2025 - 2026 FALL

Computer Engineering Graduation Design I

TITLE OF YOUR PROJECT

Project Short Title	TRACE-MES
Project Full Title	TRACE – Manufacture Execution System
Subject Classification	Manufacturing Execution Systems
Student name(s) (Team members of project)	Taha Koçak Ahmad Rashid Sadeed Egemen Çalışkan Erdoğan Başer

Abstract

Modern manufacturing facilities represent sensitive ecosystems that require rigorous monitoring and continuous optimization to maintain profitability and operational accuracy. However, currently there is significant gaps in the current solutions for these issues. Current industrial landscape is frequently hindered by legacy Manufacturing Execution Systems (MES) characterized by outdated architecture, cluttered user interfaces, and steep learning curves. These rigid systems often overwhelm stakeholders and operators, creating significant barriers to effective interaction and data utilization. Consequently, manufacturing organizations suffer from substantial operational inefficiencies—ranging from critical time loss and resource mismanagement to "data-blindness"—which can ultimately result in catastrophic damage to physical assets and overall business viability.

To address these systemic challenges, TRACE-MES provides a solution designed as a modular, user-centric platform. Distinct from traditional monolithic software, TRACE-MES prioritizes "AI-ready" data structures, facilitating integration with broader manufacturing management ecosystems, such as Product Lifecycle Management (PLM) and advanced data analysis pipelines. This interoperability is crucial for driving predictive optimization in machine operations and operations within other large-scale manufacture management software.

The system architecture is uniquely compartmentalized to serve distinct user needs: it offers a streamlined, intuitive interface for shop floor technicians requiring only basic training, while simultaneously providing granular, high-level dashboards for engineers and managers to audit and monitor complex operations. By ensuring accessibility and usability, TRACE-MES effectively bridges the gap between human operators and digital intelligence. Functionally, the system establishes a robust communication loop with shop floor machinery to automate task assignment, execute precise real-time data acquisition, and monitor operational status. This comprehensive approach ensures that critical status updates reach technicians instantly, thereby optimizing the feedback loop between physical production lines and digital management strategies.

Keywords: MES, PLM, manufacturing, AI-ready, optimization, user-friendly, resource management, modular,

Acknowledgements (if any)

We would like to express our utmost appreciation for all those who helped in the development and realization of this project. A special thanks to Mr. Hasan Başer for his assistance and valuable insights he conveyed to us in our field research at “Noktam Makine” and our senior project supervisor Dr. Abdülvahab Ömer Toprak for their valuable advice through the development process.

Table of Contents

Acronyms

CNC	Computer Numerical Control
MES	Manufacturing Execution System
PLM	Product life-cycle management
ERP	Enterprise resource planning
AI	Artificial Intelligence
RBAC	Role-based access control
TCP	Transmission control Protocol
IP	Internet protocol
SME	Small and medium-sized enterprises

Ch1: Introduction

1.1 Problem Statement

Traditional Manufacturing Execution Systems (MES) are the backbone of modern factory operations, but they are often characterized by high complexity, rigid architectures, and costly customization. A significant emerging challenge is that they create "data-silos" on the shop floor. While they collect vast amounts of real time production data (e.g., machine status, cycle times, quality metrics, operator inputs), this data is rarely structured for consumption by modern Artificial Intelligence (AI) and Machine Learning (ML) models. This "data-blind" design makes it difficult to extract predictive insights for areas like predictive maintenance, quality assurance, or production scheduling optimization. Solution This project proposes the design and development of TRACE, a novel MES platform engineered to address these challenges. It will be built on a modular, adaptive architecture, prioritizing a simple user interface and ease of maintenance. The system will feature robust, hierarchical role-based access control (RBAC) to ensure data confidentiality and a live console panel for real-time monitoring of production activities. The primary innovation of TRACE is its "AI-ready" data architecture. Unlike traditional systems, all production inputs (e.g., work orders) and outputs (e.g., cycle times, defect logs) will be structured and exported in a clean, versioned, and interpretable format (e.g., time-series data, structured logs) specifically designed for consumption by AI/ML pipelines. This system will bridge the gap between conventional shop floor management and intelligent, data-driven manufacturing.

1.2 Scope

The project's scope includes the full design, development, and testing of the TRACE web application. This covers the front-end (UI) and back-end (database, logic) for managing work orders, tracking resource status (machines/operators), and logging production counts (good/scrap). The RBAC system, live dashboard, and AI data-export module are all in-scope. The project is out-of-scope for: the development of the AI/ML models themselves (we only provide the data), direct hardware integration with PLCS/SCADA (data will be entered manually or simulated), and advanced planning/scheduling (APS) algorithms.

Ch2: Project Plan

2.1 Methodology

The TRACE-MES project follows a hybrid software development methodology that combines structured planning with iterative development principles. This approach is selected to meet the academic requirements of a graduation project while addressing the practical challenges of designing a modular Manufacturing Execution System.

During the Senior Design Project I (495) phase, a plan-driven approach is adopted, focusing on problem analysis, literature review, requirements elicitation, and high-level system design. In addition, field research is conducted at workshops equipped with CNC machines to observe real production environments and evaluate the feasibility of direct machine communication. Findings from these on-site studies directly inform system requirements, particularly those related to CNC data acquisition and bidirectional communication.

An iterative and incremental development approach, inspired by Agile methodologies, is planned for Senior Design Project II (496), where system components will be developed and refined. This hybrid methodology ensures clear documentation and realistic requirements while supporting future development and AI-supported decision-support functionalities.

2.2. Time Schedule (Gantt Chart)

You can find the Gantt chart in the appendix section A.

2.3 Deliverables List

ID	Type	Description	Due Date
			2025
D1	Document	Project Proposal	October
D2	Document	Field Study Report (CNC Workshop Observations) *(Intermediate)*	November
D3	Document	Literature Review	November
D4	Document	Requirements Analysis Draft *(Intermediate)*	November
D5	Document	Software Requirements Specification (SRS) & Intermediate Report	December
			2026
D6	Document	Graduation Design I (495) Final Report	January
D7	Document	System Architecture & High-Level Design Document *(Intermediate)*	February
D8	Software	Core MES Backend & Database Implementation	February
D9	Software	Role-Based Access Control & Work Order Management Modules	March
D10	Software	Frontend Interface & Live Monitoring Console	March
D11	Software	CNC Communication Module	April
D12	Software	AI-Ready Data Exporter	April
D13	Software	System Integration & Initial Functional Testing	April
D14	Software	System Validation, Improvements & Deployment	May
D15	Document	Working Version Presentation	May
D16	Document	Graduation Design II (496) Final Report	June
D17	Document	Final Poster & Poster Presentation	June

Ch3: Literature Review

Manufacturing Execution Systems (MES) have historically functioned as the crucial bridge between the planning layer (ERP) and the shop floor (SCADA/PLC), providing real-time visibility into the production process. However, the advent of Industry 4.0 has fundamentally altered the requirements for these systems. The literature review presented in this chapter focuses on the evolution of MES architectures towards "Next-Generation" systems, the critical importance of ERP integration for management support, and the persistent challenge of data silos in modern manufacturing.

Evolution Towards Next-Generation MES

Early research by Kletti (2015) established the foundational functions of MES, emphasizing resource allocation and status monitoring. However, as manufacturing paradigms shift, these traditional definitions are being challenged. Mantravadi and Møller (2017) provide a comprehensive overview of "Next-Generation MES," arguing that traditional, monolithic systems are often too rigid for the dynamic requirements of Industry 4.0. They posit that the future of MES lies in decentralized, modular architectures that can adapt to changing production needs without extensive re-engineering. This perspective strongly validates the TRACE project’s architectural decision to adopt a web-based, modular approach, moving away from legacy rigidity towards flexibility.

Furthermore, Pusztai, Nagy, and Budai (2024) highlight that for Small and Medium-sized Enterprises (SMEs) to remain competitive, they specifically require these scalable solutions to avoid the high implementation costs associated with traditional infrastructure.

The Imperative of Integration and Management Support

A key function of the MES is not just to control the shop floor, but to feed accurate data to upper management layers. Berić et al. (2018) investigated the implementation of ERP and MES systems as a support to industrial management. Their findings demonstrate that the seamless integration of MES with ERP is not merely technical but strategic; it significantly enhances management's ability to monitor Key Performance Indicators (KPIs) and make informed decisions. They conclude that without this effective data exchange, industrial management systems remain incomplete. TRACE addresses this by prioritizing "AI-ready" data structures and open API capabilities, ensuring that the data flow described by Berić et al. is not hindered by proprietary locks or incompatible formats.

In conclusion, the literature confirms a gap between the theoretical promise of Industry 4.0 and the practical reality of existing legacy systems. While Mantravadi and Møller (2017) identify the need for next-gen systems and Berić et al. (2018) prove the value of integration, few solutions specifically target the "data export" and "AI-readiness" required to bridge these concepts for SMEs. TRACE is designed to fill this specific void by acting as a modern, transparent intermediary that democratizes access to production data for intelligent systems.

Ch4: Requirements

4.1 Overall Description of the Project

4.1.1 Product Perspective

TRACE-MES is a modular and adaptive web-based Manufacturing Execution System (MES) designed to facilitate real-time production monitoring, work order management, and AI-ready data collection. While the platform operates as an independent and self-contained MES, it is specifically designed to coexist with higher-level enterprise systems, such as ERP solutions.

In the context of the broader industrial environment, TRACE-MES acts as the intermediary layer between high-level planning systems and physical shop-floor resources. It bridges the gap between management functions and equipment by providing a user-friendly interface, reliable communication and structured data flow.

System Interfaces and Interconnections:

The system manages several critical interfaces to ensure seamless integration between the planning phase and the execution phase:

- Shop-Floor Interface: Direct hardware integration with PLC/SCADA or physical CNC controllers is out of scope for this phase. Instead, TRACE-MES ingests shop-floor telemetry via simulated data sources (MockDataGenerator) and/or imported datasets, and it routes any control actions (e.g., STOP) to the simulation/test environment only.
- User Interface: A web-based portal serves as the primary touchpoint for operators, supervisors, and administrators, allowing for centralized control and human-centered decision support.

4.1.2 Product Functions

The major functions of TRACE-MES include:

- User authentication and role-based access control
- Job and Task order creation, assignment, and tracking
- Real-time telemetry ingestion from simulated sources and/or imported datasets
- Simulated bidirectional control routed to the simulation/test environment (no physical machine control in this phase)Live monitoring of production status
- Manual and automated quality data collection
- AI-ready data preparation and export
- Audit logging and system monitoring

4.1.3 User Characteristics

The TRACE-MES system is designed to cater to a wide range of users, from operational personnel on the production floor to management and customers receiving services. The system's target audience and the expected qualifications of these users are defined below:

Machine Operators (Workshop Personnel):

Role Description: Users who are physically present on the production floor, operate CNC machines, and enter job order start/end data and part counts into the system.

Education and Experience: Technical education at the high school or vocational college level and practical experience with the relevant machine park (CNC, etc.) are expected.

Technical Expertise: Basic digital literacy is sufficient. They are not expected to perform complex data analysis; therefore, the operator interfaces are designed to be simple, understandable, and suitable for touchscreen use.

Production Supervisors and Managers:

Role Description: They are decision-makers who create work orders, assign resources (machine/operator), and monitor the production process via the live console.

Education and Experience: Typically, individuals with a bachelor’s degree in engineering or industrial management who are proficient in production planning processes.

Technical Expertise: They must be competent in using web-based management systems, interpreting production metrics (KPIs), and using reporting tools.

System Administrators (Admin):

Role Description: They are technical personnel responsible for the overall security of the system, Role-Based Access Control (RBAC) configuration, and user account management.

Education and Experience: Personnel educated in Computer Engineering or Information Technology with experience in system administration.

Technical Expertise: Must have advanced technical knowledge in database management, network security, and system configuration.

Customers (External Users):

Role Definition: These are users who can access the system with limited permissions and view the production stages (e.g., Preparing, Processing, Completed) and estimated delivery statuses of the products they have ordered in real time.

Education and Experience: No specific education level or production experience is required for this group; they have a general user profile.

Technical Expertise: No technical expertise is required. They only need to be able to use simple status query interfaces via a standard web browser or mobile device.

4.1.4 Constraints

During the design and development process of the TRACE-MES project, there are various limiting factors that shape the scope, technical architecture, and delivery process of the project. The key constraints binding the development team are listed below:

Scope and Integration Constraints:

As stated in the project documentation, the TRACE system does not cover the development of artificial intelligence and machine learning (AI/ML) models; its focus is on providing data that these models can consume.

Ensuring full “out-of-the-box” compatibility with all legacy PLM and ERP systems on the market is outside the scope of the project.

During the project development process, only one or two major industry-standard platforms (e.g., SAP, PTC Windchill, etc.) will be used to implement sample integration scenarios.

Different or special system integrations requested by customers are planned to be developed as a company-specific “additional service” model once the product is commercialized and made available for use.

Hardware Constraints:

Direct hardware integration (physical cabling) with PLC/SCADA systems in a real industrial environment is outside the scope of the project. CNC machine data will be provided during the development process through manual input or using software simulation tools.

Time Constraints:

The project is strictly tied to the university's academic calendar. The design (495) and final implementation (496) phases must be completed by the specified dates, which requires prioritizing features (MVP).

Legal and Security Standards:

Compliance with KVKK and legal data privacy standards is mandatory due to the personnel and customer order data stored in the system.

Role-Based Access Control (RBAC) must be restricted, particularly to ensure that customer roles can only view their own order data.

Economic Constraints:

Within the student project budget, the use of open-source technologies that do not require licensing costs should be preferred.

4.1.5 Assumptions and Dependencies

When determining the requirements and making design decisions for the TRACE-MES project, it was assumed that the following factors would remain valid throughout the project. Any change in these factors may require a reassessment of the system requirements.

Assumptions:

Representativeness of Simulation Data: Since real hardware integration is out of scope, it is assumed that the CNC simulation data (cycle times, error codes, etc.) to be used in the development and testing phases adequately represent real-world scenarios and data formats with sufficient accuracy.

Network Infrastructure: It is assumed that a stable internet or intranet connection is available at the production facilities where the system will be used and on the customer side to provide uninterrupted access to web-based interfaces.

Client Hardware: It is assumed that users (operators, managers, and customers) have up-to-date devices (PC, tablet, or industrial panel) capable of running modern web browsers (Chrome, Firefox, Edge, etc.) to use the system.

End-User Competence: It is assumed that workshop personnel (operators) are familiar with basic user GUI, touchscreen interactions and are literate.

Dependencies:

Third-Party Software and Libraries: The project relies on open-source libraries (e.g., React, Django, PostgreSQL) to reduce development costs. These technologies must continue to be supported and must not contain critical security vulnerabilities.

ERP Integration Documentation: It depends on access to the developer documentation and test environments (Sandbox API) of large-scale ERP systems (e.g., SAP or PTC Windchill etc.) for which an integration demo will be performed within the scope.

Academic Calendar and Advisor Approval: The progress of the project is directly dependent on the academic calendar set by the university (courses 495 and 496) and the feedback provided by the project advisor during interim submissions.

4.2 Specific Requirements

4.2.1 External Interface Requirements

The TRACE-MES system interacts with three primary categories of external entities: Human Users (via Web Interface), Industrial Hardware (via CNC/IoT Interface), and External Analysis Systems (via AI-Ready Data Export). The specific requirements for these interfaces are detailed below.

1. Shop-Floor & Management User Interfaces (Web Portal)

Name of item: Web-Based User Dashboard & Control Panel

- Description of purpose: To allow operators to enter production data, supervisors to manage work orders, and admins to configure the system. It serves as the primary visualization tool for real-time monitoring.
- Source of input: Human users (Operators, Supervisors, Admins) via keyboard, mouse, or touchscreen devices.
- Destination of output: Web Browser (Chrome, Edge, etc.) running on PC, Tablet, or Industrial Panels.
- Valid range, accuracy, and/or tolerance:
- Production Counts: Integers ≥0.
- Timestamps: Accurate to the second (UTC).
- Screen formats/organization:
- Responsive web design implementing a hierarchical view based on RoleType (e.g. ADMIN, SUPERVISOR, OPERATOR).
- Live Console: Displays dynamic cards for each machine showing MachineStatus (e.g. IDLE, ACTIVE, ALARM).

Data formats:

- Inputs are validated JSON objects sent via HTTPS (REST API).
- Passwords are hashed before storage (passwordHash).
- Command formats: Button clicks trigger API calls (e.g., POST /api/execution/start, POST /api/quality/scrap).

- End messages: HTTP Status Codes (200 OK, 400 Bad Request, 401 Unauthorized) accompanied by user-friendly toast notifications (e.g., "Work Order Started Successfully").

2. Telemetry Simulation & Data Import Interface

- **Name of item: Telemetry Data Ingestion (Simulation / Import)**
- **Description of purpose:** To acquire time-series telemetry data for monitoring, anomaly labeling, and AI-ready export without requiring direct industrial hardware integration. This interface bridges the gap between the data sources (simulation or files) and the MES logic.

  Source of input:

- MockDataGenerator service (simulated CNC telemetry stream).
- Offline data import files (CSV/JSON) generated manually or from external sources.
- **Destination of output:** TRACE-MES Backend (Telemetry Ingestion Service).

  Units of measure:

- **Spindle Speed:** RPM (Float)
- **Feed Rate:** mm/min or unit/min
- **Temperature:** Celsius (°C)
- **Axis Position (X, Y, Z):** Millimeters (Float)
- **Timing:** Near real-time ingestion from the simulator. The system targets an update rate similar to a real shop-floor stream (e.g., ~1 Hz) when the simulation mode is active.

  Data formats:

- The TelemetryPacket JSON structure includes: { timestamp, machineId, spindleSpeedRpm, axisX, axisY, axisZ, temperature }.

  Protocols supported (prototype scope):

- HTTPS REST API for batch data import (e.g., POST /api/telemetry/import).
- WebSocket or Server-Sent Events (SSE) for live simulator streaming.
- Error handling:

- If the simulated stream stops or no data is received within a defined threshold, the system automatically marks the machine status as OFFLINE or UNKNOWN and logs an ingestion warning.
- Import failures return explicit validation errors with details (e.g., "Malformed timestamp", "Missing machineId").

3. AI-Ready Data Export Interface

Name of item: AI Data Exporter

- Description of purpose: To provide clean, structured, and versioned datasets for external Machine Learning models, enabling predictive maintenance and optimization analysis.
- Source of input: TRACE-MES Database (Aggregated from TelemetryPacket, WorkOrderExecution, and ProductionLog).
- Destination of output: File System (Downloadable files) for Data Scientists or ML Pipelines.

Data formats:

- Supported output formats include CSV, JSON, and PARQUET as defined in DataExportJob.
- Files are accompanied by a schemaVersion to ensure compatibility with external parsers.
- Relationships to other inputs/outputs: The export links AnomalySnapshot data with ScrapLog reasons, allowing external AI to correlate machine telemetry with specific defect types.
- End messages: Upon completion, the system updates the DataExportJob status to "COMPLETED" and generates a downloadable FileAsset link.

4.2.2 Functional requirements

The functional requirements for the TRACE-MES system are categorized into five core modules: **Identity & Access Management**, **Work Order Management**, **IoT & CNC Integration**, **Production Execution & Quality**, and **AI-Ready Data Export**. These requirements define the fundamental actions, input validations, sequence of operations, and error handling mechanisms of the system.

1. **Identity and Access Management (RBAC):** This module manages system security, user authentication, and role-based access control policies.

- **FR-IAM-01 (Authentication):** The system shall authenticate users via a secure login interface using a unique username and password. Passwords shall be stored in the database using a strong cryptographic hash function (e.g., PBKDF2 or Argon2).

- **FR-IAM-02 (Session Management):** The system shall validate active user sessions and strictly enforce an auto-logout mechanism after a configurable period of inactivity (e.g., 15 minutes) to ensure security in the shop floor environment.
- **FR-IAM-03 (Role-Based Authorization):** The system shall restrict access to functions based on the assigned user role:
  - **Operators:** Limited to "Live Execution" tracking and "Manual Quality Entry".
  - **Supervisors:** Authorized for "Work Order Management" and "Live Console" monitoring.
  - **Admins:** Full access to system configuration and user management.

2. **Work Order Management & Planning:** This module governs the creation, assignment, and lifecycle management of production orders.

- **FR-WO-01 (Work Order Creation):** The system shall allow authorized users to create Work Orders by defining a unique WorkOrderCode, selecting a MaterialDefinition (Part Number), setting a PlannedQuantity, and assigning a Priority level.
- **FR-WO-02 (Input Validity Checks):**
  - The system shall validate that the PlannedQuantity is a positive integer greater than zero.
  - The system shall prevent the deletion of any Work Order that has associated ProductionLogs or TelemetryPackets to maintain data integrity.
- **FR-WO-03 (State Machine Sequence):** The system shall enforce the following strict sequence of operations for Work Order status transitions:
  - PENDING IN_PROGRESS: Triggered only when the Operator initiates the start command.
  - IN_PROGRESS PAUSED: Triggered by a manual pause or machine halt.

  - IN_PROGRESS COMPLETED: Triggered automatically or manually when the production target is met.

- **FR-WO-04 (Resource Assignment):** The system shall support the assignment of Work Orders to specific Machines and Shifts, enabling precise shift-based performance reporting.

3. **IoT & CNC Telemetry Integration**: This module handles the acquisition of shop-floor telemetry data and control signaling without direct industrial hardware integration, utilizing simulation streams to validate system logic.

- **FR-TEL-01 (Simulation Stream Ingestion):** The system shall ingest real-time telemetry data from a simulated source (MockDataGenerator) and create TelemetryPacket records containing at minimum: timestamp, machineId, spindleSpeedRpm, axisX, axisY, axisZ, and temperature.
- **FR-TEL-02 (Batch Import):** The system shall support importing historical or sample telemetry datasets via a REST endpoint using CSV or JSON formats, validating schema correctness before persistence.
- **FR-TEL-03 (Abnormal Situation - Data Loss):**
  - **Detection:** The system shall detect a data loss event if no telemetry packet is received for a configured threshold (e.g., 3 seconds).
  - **Recovery:** Upon detection, the system shall automatically transition the machine status to OFFLINE and log the incident.
- **FR-TEL-04 (Bidirectional Control Prototype):**
  - The system shall provide a user interface for authorized Supervisors to issue control commands (e.g., "EMERGENCY STOP", "RESET ALARM").
  - **Constraint:** To ensure safety during the development phase, these commands shall be routed to the **Simulation Environment** or **Test Interface** only. The system shall verify the command execution on the digital twin (simulator) without interfacing with live high-voltage industrial equipment.

4. **Production Execution & Quality Control**: This module tracks the actual manufacturing process and manages quality data inputs.

- **FR-QC-01 (Execution Tracking):** The system shall generate a WorkOrderExecution record for each production run, logging precise StartedAt and EndedAt timestamps to calculate the actual Machine Runtime and OEE metrics.
- **FR-QC-02 (Manual Quality Entry):** The system shall provide a user interface for operators to input Good Quantity and Scrap Quantity at the completion of a job or shift.
- **FR-QC-03 (Scrap Validation & Categorization):**
    - If Scrap Quantity is greater than zero, the system shall require the operator to select a mandatory DefectCategory (e.g., Dimensional Error, Broken Tool).
    - The system shall validate that the total reported quantity (Good + Scrap) does not exceed the physically possible production rate for the elapsed time.
- **FR-QC-04 (Context-Aware Anomaly Snapshot):** Upon the submission of a scrap record, the system shall automatically retrieve and tag the telemetry data from the preceding time window (e.g., last 5 minutes) as an AnomalySnapshot to serve as labeled training data for AI models.

5. **AI-Ready Data Export:** This module transforms raw operational data into structured, machine-learning-ready datasets.

- **FR-AI-01 (Data Aggregation & Conversion):** The system shall aggregate data from disparate sources—TelemetryPackets (IoT), WorkOrderExecution (Context), and ProductionLogs (Labels)—to generate a unified, time-series dataset.
- **FR-AI-02 (Export Formats):** The system shall support data export in industry-standard formats, including CSV, JSON, and PARQUET, to ensure compatibility with external AI/ML pipelines.
- **FR-AI-03 (Versioning & Metadata):** The system shall append metadata to exported files, including SchemaVersion and SensorConfigVersion, to ensure that AI models can correctly interpret data across different sensor calibration periods.
- **FR-AI-04 (Error Handling):** In the event of an export failure (e.g., storage limit reached), the system shall mark the job status as FAILED, log the specific error, and notify the user to retry the operation.

4.2.3 Quality Requirement (HW, OS, Database, programming environment, performance,security, reliability, etc)

a) Specific Hardware Requirements

- **QR-HW-01 (Server Specifications):** The backend server hosting the Django application and database shall utilize a standard hosting environment with sufficient resources (e.g., minimum 2 vCPU cores and 4GB RAM) to handle concurrent WebSocket connections and telemetry processing.
- **QR-HW-02 (Client Terminals):** The system shall be accessible via standard personal computers (desktops/laptops) or mobile devices (tablets) available in a typical office or workshop environment.
- **QR-HW-03 (Connectivity):** The host machine must possess a stable network interface to support web-based telemetry ingestion (e.g., REST/WebSocket) from simulated sources and to upload/import datasets reliably.

b) Specific Operating System Requirements

- **QR-OS-01 (Server Environment):** The production environment shall be compatible with Linux-based operating systems (e.g., Ubuntu, Debian) to ensure optimal performance for the Python/Django backend.
- **QR-OS-02 (Client Independence):** The client-side application shall be platform-agnostic, functioning correctly on any operating system (Windows, Linux, macOS, Android) capable of running a modern web browser (Chrome, Edge, Firefox).

c) Database Requirements

- **QR-DB-01 (Data Integrity):** The system shall utilize PostgreSQL as the primary Relational Database Management System (RDBMS) to ensure ACID compliance, guaranteeing the reliability of production logs and work order status updates.
- **QR-DB-02 (Indexing):** The database schema shall be optimized with appropriate indexes on timestamp columns to support efficient querying of historical telemetry data for export operations.

d) Specific Programming Environment Requirement

- **QR-PE-01 (Backend Framework):** The server-side logic shall be developed using Python (Django) to leverage its ecosystem for web APIs, data validation, time-series processing, and AI-ready dataset preparation/export.
- **QR-PE-02 (Frontend Framework):** The user interface shall be developed using React utilizing the Ant Design component library to ensure a modular, responsive, and consistent user experience.
- **QR-PE-03 (Version Control):** Source code shall be managed using a version control system (Git) to facilitate collaborative development and change tracking.

e) Performance Requirement

- **QR-PERF-01 (System Latency):** The latency between the reception of a telemetry packet and its visualization on the "Live Console" shall be minimized (target < 2 seconds) to ensure near real-time monitoring capabilities.
- **QR-PERF-02 (Sampling Rate):** The system shall be capable of processing incoming telemetry data at a minimum frequency of 1 Hz (1 record per second) per connected machine.
- **QR-PERF-03 (Responsiveness):** The web interface shall load critical dashboards within a reasonable time frame to maintain user engagement and operational efficiency.

f) Security Requirements

- **QR-SEC-01 (Password Security):** User passwords shall never be stored in plain text; they must be hashed using secure algorithms (e.g., PBKDF2) provided by the Django authentication system.
- **QR-SEC-02 (Access Control):** The system shall strictly enforce the defined Role-Based Access Control (RBAC) policies, ensuring that operators cannot access administrative configurations.
- **QR-SEC-03 (Session Security):** API endpoints shall be protected against unauthorized access, requiring valid session tokens for all non-public operations.

g) Availability and Reliability Requirement

- **QR-REL-01 (Connection Recovery):** The system shall detect disconnections from the telemetry source (e.g., simulator/adapter) and attempt to reconnect automatically without requiring a full system restart, while logging the incident and updating machine status accordingly.
- **QR-REL-02 (Data Persistence):** The system shall ensure that all committed production data (Good/Scrap counts) is persistently stored in the database to prevent data loss during browser refreshes or minor network interruptions.

Ch5: Design

5.1 Software Architecture

The TRACE-MES system follows a **Layered Architecture** pattern, specifically designed to decouple the user interface from the complex business logic and data processing layers. This separation of concerns allows for independent evolution of the frontend and backend components, facilitating maintenance and future scalability. The architecture is web-based, ensuring accessibility across various devices (PCs, tablets) without requiring local installation.

5.1.1 Actors

TRACE-MES is designed to be used by multiple stakeholders on the shop floor while also integrating with external systems that either provide data or consume it.

Human Actors

- **Operator:** The main shop-floor user. Operators access the system via tablets or terminals to monitor the current machine state, review assigned work orders, and enter production outputs (good quantity and scrap details).
- **Supervisor:** Responsible for operational oversight. Supervisors use dashboards to track overall performance, review shop-floor trends, adjust work-order priorities, and intervene when production issues occur.
- **Administrator:** Handles configuration and system governance. Administrators manage user accounts (RBAC), register machines, and maintain global settings required for stable operation.
- **Customer:** An external user with limited read-only access to view the status and estimated delivery of their specific orders.

System Actors

- **MockDataGenerator (Simulation Engine):** Represents the CNC machine in the development and testing phase. It generates continuous raw telemetry streams (Speed, Temperature, Axis Coordinates) to validate the system's ingestion capabilities without physical hardware risks.
- **API Client (External System):** Represents downstream analytics systems or automated scripts. This actor interacts with TRACE-MES through the Data Export API using an API Key to retrieve structured, historical datasets for offline analysis.

5.1.2 Constraints

The architecture and technology choices are guided by operational realities and the project's specific scope boundaries.

- **Low Latency Display:** Live machine telemetry must appear on operator screens with minimal delay (< 2 seconds) to ensure real-time situational awareness.
- **Context-Aware Data Storage:** Telemetry is only valuable for AI if stored with context. Sensor readings must be tightly coupled with the active WorkOrder and Operator ID.
- **Unstable Connectivity Tolerance:** The system must handle brief network disconnects between the client and server without data loss, utilizing local state management where appropriate.
- **Simulation-First Design:** Due to the hardware "Out-of-Scope" constraint, the architecture is optimized to ingest data from the MockDataGenerator service over HTTP/WebSocket, simulating an industrial protocol adapter.

5.1.3 Software Architecture

The system utilizes a modern technology stack comprising a **React** Single Page Application (SPA) for the frontend and a **Django (Python)** backend.

- **Presentation Layer (Frontend):** Handles UI rendering and user interactions using React and Ant Design. It communicates with the backend via REST API for transactions and WebSockets for live data.
- **Application Layer (Backend):**
  - **Core API:** Manages business logic (Work Orders, Users, Quality Logs).
  - **IoT Service:** A dedicated module listening for incoming telemetry from the Simulator.
  - **Channel Layer:** Uses **Redis** and **Django Channels** to broadcast real-time updates to connected clients.
- **Data Layer: PostgreSQL** is used for persistent relational data storage (ACID compliance), while Redis serves as a high-speed message broker.

5.2 Views

5.2.1 Logical View

he Logical View describes the static structure of the system, defining the classes, their attributes, and the relationships between them. This view is essential for understanding how data is modeled and interconnected within the database.

Key entities in the TRACE-MES Logical View include:

- **WorkOrder:** The central entity linking production targets to materials.
- **Machine:** Represents the equipment (or simulator) being monitored.
- **TelemetryPacket:** High-frequency time-series data points.
- **AnomalySnapshot:** A specialized dataset linking scrap events to telemetry history for AI training.

[See APPENDIX B for the detailed Class Diagram]

5.2.2 Process View

The Process View captures the dynamic behavior of the system, illustrating how objects interact to fulfill specific functional requirements. It describes the flow of control and data between actors and system components.

Use Case Overview: The system supports critical operational use cases categorized by actor roles:

- **Production Execution:** Operators can *Start*, *Pause*, *Resume*, and *Complete* executions.
- **Quality Control:** Includes logging *Scrap Quantity* and capturing *Anomaly Snapshots* automatically.
- **Monitoring:** Supervisors utilize the *Live Console* to view real-time telemetry and machine status.
- **Data Export:** Both Supervisors and API Clients can request AI-ready datasets.

  [See APPENDIX C for the detailed Use Case Diagram]

Sequence of Operations: To illustrate the timing and message exchange, specific flows are mapped in the following diagrams:

1. **User Authentication (SD-01):** Shows the login flow returning a JWT token and the heartbeat mechanism that enforces auto-logout after 15 minutes of inactivity.
2. **Work Order Management (SD-02):** Details the creation of a work order by a Supervisor and its assignment to a machine and operator.
3. **Production Execution (SD-03):** Illustrates the state transitions (PENDING → IN_PROGRESS → COMPLETED) triggered by Operator actions.
4. **Quality Entry & Anomaly Capture (SD-04):** Demonstrates how entering a scrap quantity triggers the system to fetch the last 5 minutes of telemetry and save it as an AnomalySnapshot.

5. **Live Telemetry Monitoring (SD-05):** Shows the real-time WebSocket stream from the MockDataGenerator to the Dashboard, including connection loss detection (3-second threshold).
6. **AI-Ready Data Export (SD-06):** Describes the asynchronous background job that aggregates data into PARQUET/CSV formats for download by Supervisors or API Clients.

[See APPENDIX D for the detailed Sequence Diagrams]

5.2.3 Deployment View

The Deployment View describes how the software components are mapped to the underlying hardware or hosting infrastructure.

- **Server Side:** The Django Backend, PostgreSQL Database, and Redis Broker are containerized (using Docker) to ensure consistency across development and production environments. They run on a Linux-based host.
- **Client Side:** The React application is served as static assets to the client's browser.
- **Simulation Environment:** The MockDataGenerator runs as a separate service (potentially on a separate container or thread), pushing data to the Backend's exposed endpoints to mimic an external machine over the network.

Appendix A

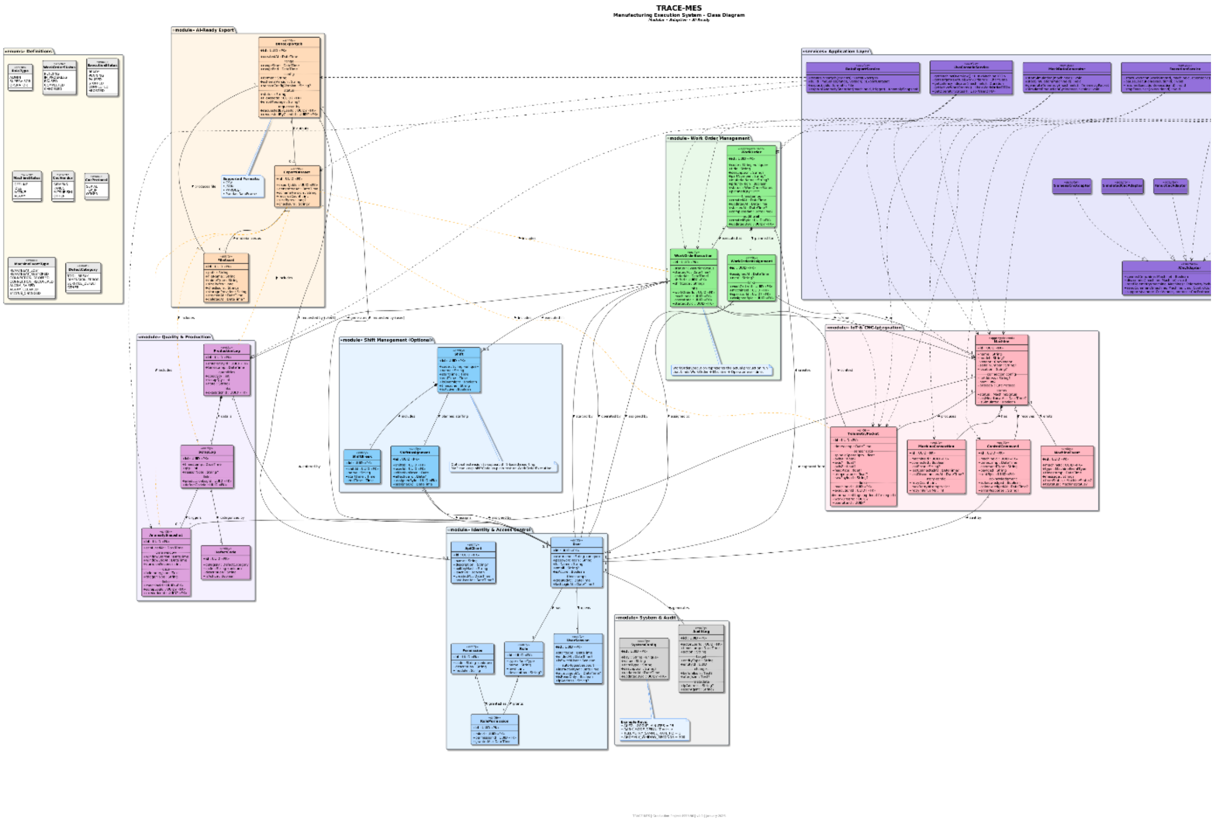
Project schedule Gannt chart.

Task / Month	Oct 2025	Nov 2025	Dec 2025	Jan 2026	Feb 2026	Mar 2026	Apr 2026	May 2026
Fall Semester (495)								
Project Proposal	**Proposal**							
Methodology & Literature Review		**Research**						
Requirements Elicitation & Analysis			**Analysis**					
Intermediate Report			**Report**					
Design Phase (Architecture, DB, UI/UX)				**Design**				
495 Final Report				**Report**				
Spring Semester (496)								
Core MES Development (DB & Backend)					**Backend**			
RBAC, Live Console & Frontend						**Frontend**		
AI-Ready Data Exporter							**Data Export**	
Testing & Validation							**Testing**	
Working Version Presentation							**Present**	
Improvement & Deployment								**Deploy**
Final Poster Presentation								**Poster**

Fall Semester (495)
Spring Semester (496)

APPENDIX B

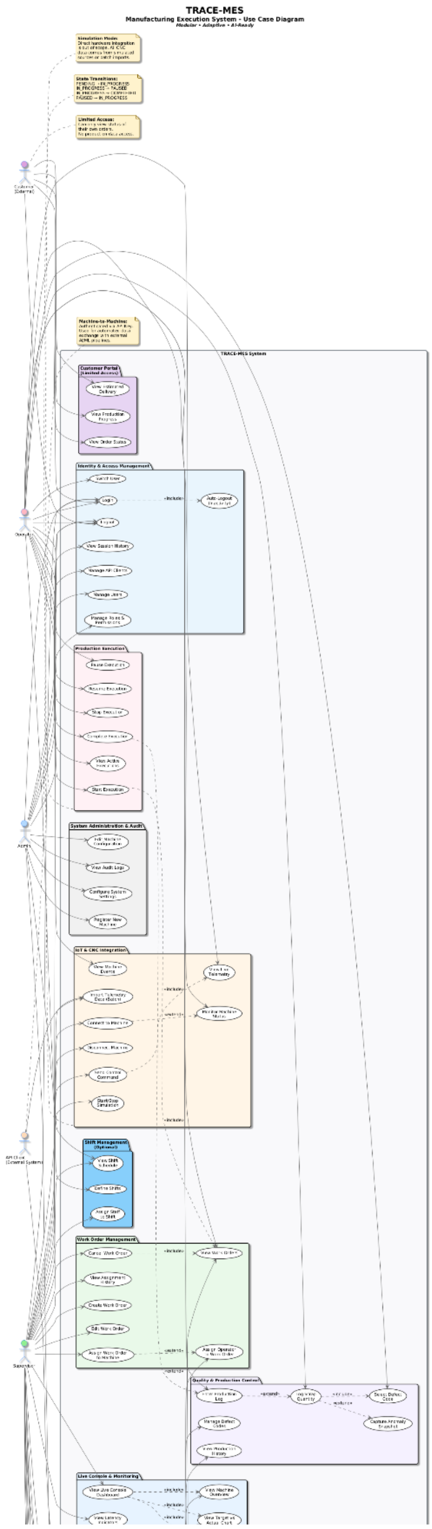
Class Diagram (.svg files can be accessed in the google drive link below for more detailed information)



https://drive.google.com/drive/folders/1Q8PUeD8pMhljy3K_EhYDOG4PZ1t_TkNS

APPENDIX C

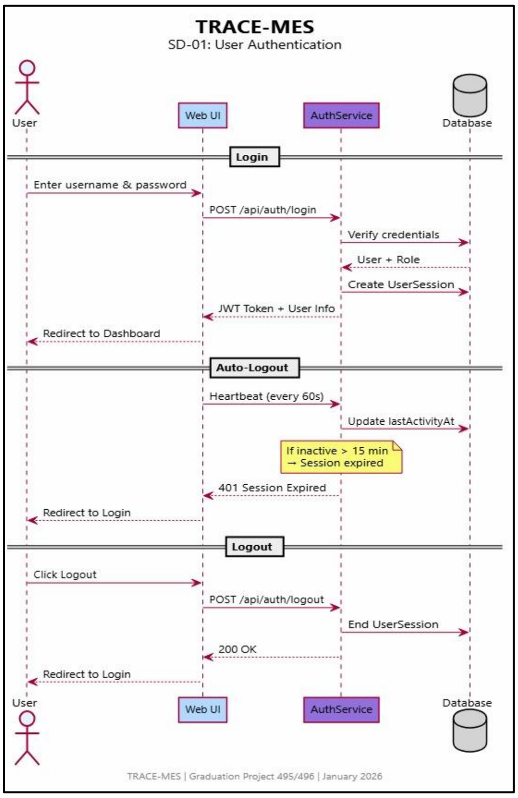
Use Case Diagram (.svg files can be accessed in the google drive link below for more detailed information)

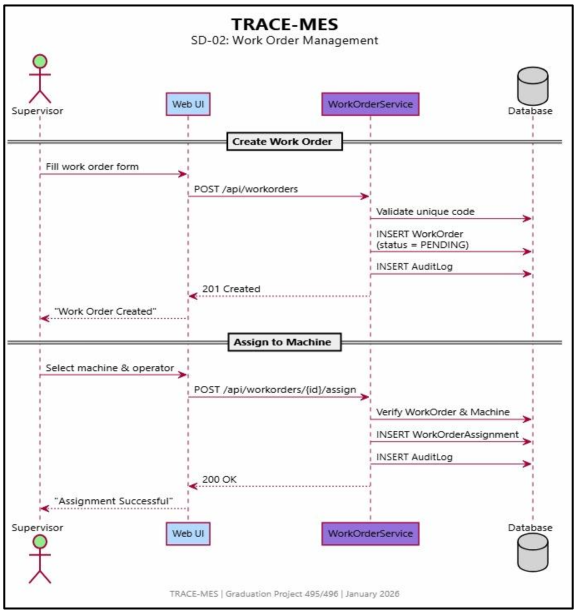


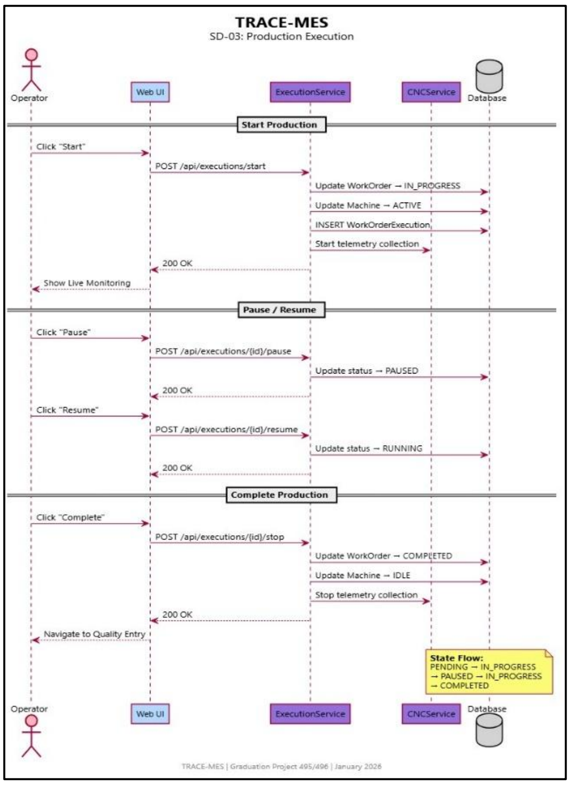
https://drive.google.com/drive/folders/1Q8PUeD8pMhljy3K_EhYDOG4PZ1t_TkNS

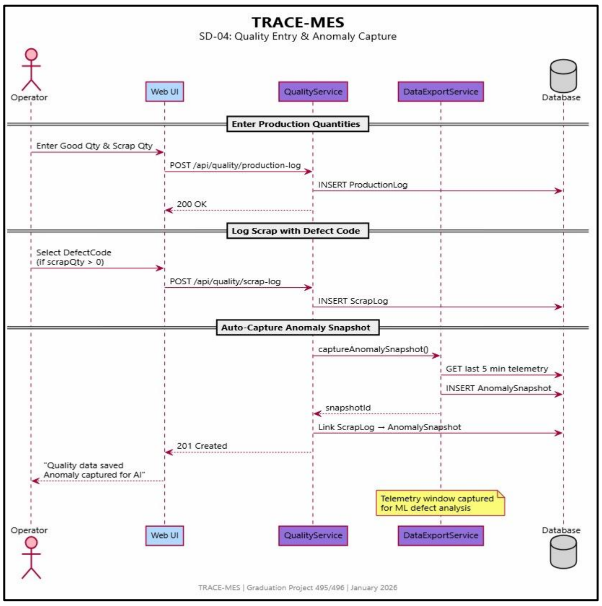
APPENDIX D

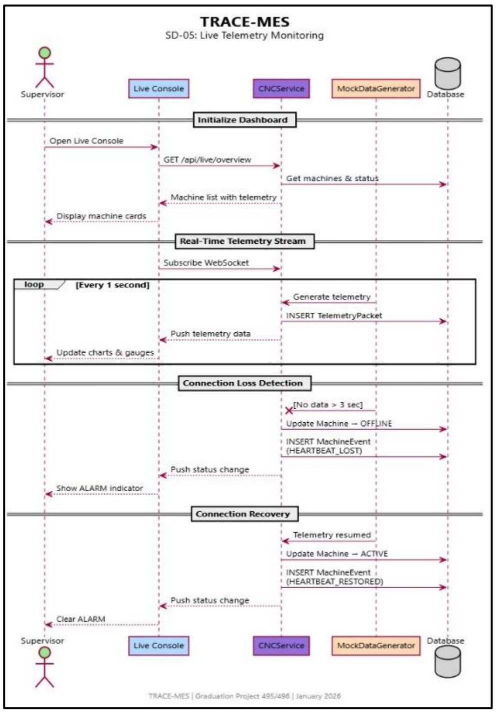
Sequence Diagram (.svg files can be accessed in the google drive link below for more detailed information)

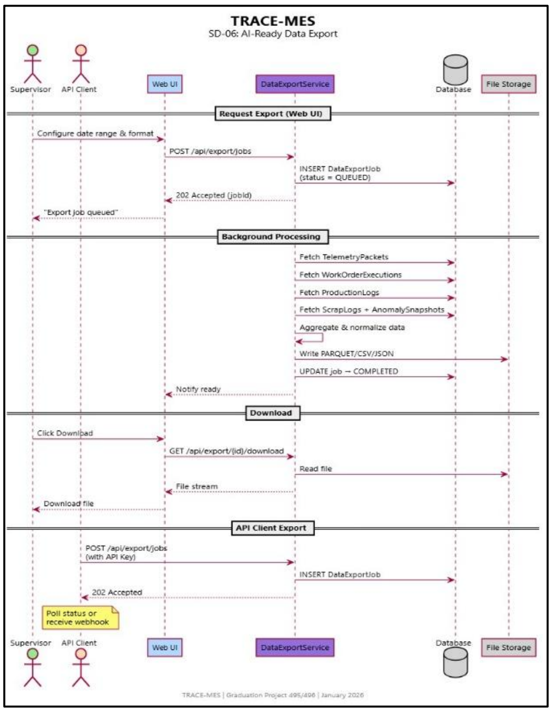












https://drive.google.com/drive/folders/1Q8PUeD8pMhljy3K_EhYDOG4PZ1t_TkNS

References

Dieguez, T., Machado, J. and Silva, F. (2025). 'Reviewing Manufacturing Execution System in Industry 4.0: A Global Approach', *EAI Endorsed Transactions on Industrial Networks and Intelligent Systems*, 12(4), pp. 1-15.

IJIRMPS (2022). 'Integrating Deep Learning with MES-ERP Systems for Real-Time Production Optimization and Resource Allocation in Smart Manufacturing', *International Journal of Innovative Research in Engineering & Management*, 9(3), pp. 45-52.

Innorobix (2025). *Real-Time Job Tracking via MES Integration with CNCs*. [online] Available at: https://www.innorobix.com/real-time-job-tracking [Accessed 10 Jan. 2026].

Kletti, J. (2015). *Manufacturing Execution Systems (MES)*. Berlin: Springer.

Pusztai, L.P., Nagy, L. and Budai, I. (2024). 'Manufacturing Execution System Application within Manufacturing Small–Medium Enterprises towards Key Performance Indicators Development', *Sustainability*, 16(7), p. 2974.